

Signale

Signale ermöglichen es, einen Prozess *von aussen* zu unterbrechen, wie ein *Interrupt*. Unterbrechen des gerade laufenden Prozesses/Threads, Auswahl und Ausführen der *Signal-Handler-Funktionen*, Fortsetzen des Prozesses. Werden über ungültige Instruktionen oder Abbruch auf Seitens Benutzer ausgelöst. Jeder Prozess hat pro Signal einen Handler.

Die 3 Default-Handler für Signale

Mit kill kann eines dieser Signale gesendet werden:

Abnormal-Terminate-Handler: beendet das Programm und erzeugt Core Dump (für Programmfehler)

SIGFPE (Fehler in arithmetischen Operation), *SIGILL* (Ungültige Instruktion), *SIGSEGV* (Ungültiger Speicherzugriff), *SIGSYS* (Ungültiger Systemaufruf)

Terminate-Handler beendet das Programm (Prozess abbrechen)

SIGTERM (Normale Anfrage an den Prozess, sich zu beenden), *SIGINT* (Nachdrücklichere Aufforderung an den Prozess, sich zu beenden), *SIGQUIT* (Wie *SIGINT*, aber anormale Terminierung), *SIGABRT* (Wie *SIGQUIT*, aber vom Prozess an sich selber), *SIGKILL* (Prozess wird "abgewürgt", kann nicht verhindert werden)

Ignore-Handler ignoriert das Signal

SIGTSTP (Versetzt den Prozess in den Zustand stopped, ähnlich wie waiting), *SIGSTOP* (Wie *SIGTSTP*, aber kann nicht ignoriert oder abgefangen werden), *SIGCONT* (Setzt den Prozess fort)

int sigaction(int signal, struct sigaction *new, struct sigaction *old):

Definiert Signal-Handler für signal, wenn new $\neq$ 0. (Eigene Signal-Handler definiert via sigaction struct: sa_handler: Zu callende Funktion, sa_mask: Blockierte Signale während Ausführung, bearbeitet nur durch sig*set()-Funktionen: sigemptyset, sigfillset, sigaddset, sigdelset, sigismember)

sigaction = transp. Datenstruktur, sigset_t = opaque Datenstr. + implementationsabhängig